

ENS2025

Simulador de ensamblador IEEE 694

Manual de usuario

Índice

1.	Introducción	4
2.	Arquitectura de la máquina virtual	4
3.	Configuración del sistema	6
4.	Estructura del código ensamblador	8
5.	Modos de direccionamiento	9
6.	Juego de instrucciones	13
7.	Etiquetas	22
8.	Directivas del ensamblador	22
9.	Detección de errores en el código fuente	23
10.	Interfaz de línea de comandos	23
11.	Interfaz Gráfica de Usuario	28
12.	Tablas de referencia	37

1. Introducción

ENS2025 es una herramienta dedicada al ensamblado y simulación de código ensamblador del estándar IEEE 694. Se trata de una mejora de herramientas ya existentes como ENS96 y ENS2001; en especial de esta última, que ya ofrecía una arquitectura de máquina virtual versátil y una interfaz de usuario, tanto por línea de comandos como de manera gráfica, superiores a sus predecesores. ENS2025 busca mejorar esta experiencia de usuario, añadiendo funcionalidades y facilitando el uso de las ya existentes.

El propósito de este manual es guiar al usuario de manera rápida y sencilla en el uso de la herramienta, dejando para la memoria de desarrollo de la misma los fundamentos técnicos y teóricos.

Además, este manual contiene tanto la guía de uso de la interfaz de línea de comandos como la interfaz gráfica, ya que ambas comparten el mismo sistema subyacente.

La herramienta se distribuye en tres directorios: uno para Windows, otro para MacOS y otro para Linux. Cada directorio contiene un fichero `.jar` con la versión de interfaz de línea de comandos, una carpeta con la versión de interfaz gráfica, y un acceso directo al script de lanzamiento de la versión de interfaz gráfica. Se incluye también el manual de la herramienta y el código fuente de la misma.

La versión de interfaz de línea de comandos requiere Java 11 instalado¹. La interfaz gráfica, por el contrario, no requiere la presencia de Java en el equipo. En ningún caso se requiere un proceso de instalación o desinstalación propio para la herramienta.

2. Arquitectura de la máquina virtual

El sistema simula una arquitectura estructurada de la siguiente forma:

- Procesador con tamaño de palabra de 16 bits.
- Memoria de 2^{16} (65536) palabras de 16 bits.
- Banco de 16 registros, con un código de identificación de 4 bits indicado por la tabla 1:
 - **R0 a R9:** Registros de propósito general. Aunque el sistema da libertad al usuario para modificar el contenido de los registros como vea conveniente, el contenido de estos diez registros no varía salvo que el usuario lo decida, mientras que los otros seis son empleados de manera implícita por el sistema y su contenido puede ser manipulado en cualquier momento.
 - **A:** Registro acumulador. Almacena el resultado de las operaciones aritmético-lógicas.

¹En teoría, está programado para ser compatible con Java 8; pero solamente se garantiza a partir de Java 11.

- **SR:** Registro de estado. Almacena el conjunto de biestables del sistema en los seis bits menos significativos. El resto de bits del registro no sufren cambios cuando se modifica un biestable.
 - **IX, IY:** Índices para el direccionamiento relativo.
 - **SP:** Puntero de pila. Indica la primera posición **libre** por encima de la pila.
 - **PC:** Contador de programa. Indica la posición de memoria de la primera palabra de la siguiente instrucción a ejecutar.
- Biestables de estado contenidos en los 6 bits menos significativos del registro de estado, asignados según la tabla 2:
 - **Z:** Cero. Se activa si el resultado de una operación es 0, se desactiva en caso contrario.
 - **C:** Acarreo. Se activa si el resultado excede los 16 bits de longitud, se desactiva en caso contrario.
 - **V:** Desbordamiento. Se activa si el resultado excede el rango de representación de 16 bits en complemento a 2; es decir, de -32768 a 32767. Se desactiva en caso contrario.
 - **P:** Paridad. Se activa si el resultado tiene un número impar de bits a 1, se desactiva en caso contrario.
 - **S:** Signo. Se activa si el resultado es negativo (si bit más significativo es 1), se desactiva en caso contrario.
 - **H:** Fin de ejecución. Se activa tras leer una instrucción **HALT**.
 - Entrada y salida por consola de números enteros de 16 bits y texto, tanto caracteres individuales como cadenas de ellos.
 - Capacidad de operar sobre enteros con signo de 16 bits.
 - Generación de excepciones durante la ejecución:
 - **Instrucción no implementada:** Se produce cuando se intenta decodificar un código de operación que no corresponde con ninguna instrucción definida, o bien cuando la instrucción tiene uno o dos operandos y se ve asociada con un modo de direccionamiento incompatible.
 - **División entre cero:** Se produce cuando el segundo operando de instrucción de división (DIV) o módulo (MOD) es cero.
 - **Final de memoria alcanzado:** Se produce cuando, tras ejecutar una instrucción (o durante la decodificación de una), el registro PC desborda el límite de memoria y vuelve a valer 0.
 - **Punto de ruptura encontrado:** Se produce cuando, durante la ejecución en modo de depuración, se intenta decodificar y ejecutar una instrucción que contiene un punto de ruptura en alguna de sus palabras.
 - **Contador de programa invade la zona de pila (opcional):** Se produce cuando el PC toma un valor comprendido entre la dirección inicial de la pila y el valor actual del SP. Se puede activar o desactivar en la configuración.
 - **Puntero de pila invade la zona de código (opcional):** Se produce cuando el SP toma un valor comprendido entre las direcciones inicial y final del código. Se puede activar o desactivar en la configuración.

- **Desbordamiento de pila (opcional):** Se produce cuando, como resultado de una inserción, el SP cruza el umbral entre la última dirección de memoria y la primera. Se puede activar o desactivar en la configuración.
- **Subdesbordamiento de pila (opcional):** Se produce cuando, como resultado de una operación (extracción, asignación de valor...), el SP tiene un valor “anterior” a la dirección inicial de la pila (mayor si la pila crece negativamente, menor si crece positivamente). Se puede activar o desactivar en la configuración.

Registro	Código	Bits	Registro	Código	Bits
R0	0	0000	R8	8	1000
R1	1	0001	R9	9	1001
R2	2	0010	A	10	1010
R3	3	0011	SR	11	1011
R4	4	0100	IX	12	1100
R5	5	0101	IY	13	1101
R6	6	0110	SP	14	1110
R7	7	0111	PC	15	1111

Tabla 1: Tabla de códigos de registro.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	H	S	P	V	C	Z

Tabla 2: Tabla de posicionamiento de biestables en el registro SR.

3. Configuración del sistema

ENS2025 tiene una serie de parámetros de configuración que modifican su comportamiento bajo ciertas circunstancias. Dichos parámetros son los siguientes:

- **Comprobación de invasión de pila:** Indica si, cada vez que cambie el valor del PC, se debe comprobar primero si la nueva dirección está dentro de los límites de la zona de pila.
- **Comprobación de invasión de código:** Indica si, al modificar el SP, se debe comprobar primero si la nueva dirección está dentro de los límites de la zona de código.
- **Comprobación de reinicio de registros:** Indica si, al usar alguno de los métodos de la clase `Simulator` para ejecutar código, se debe comprobar si el biestable H está en alto a fin de reiniciar primero el banco de registros.
- **Comprobación de overflow:** Indica si, al incrementar la pila, se debe comprobar que el SP no pase de 65535 a 0 (si es creciente) o de 65535 a 0 (si es decreciente).

- **Comprobación de underflow:** Indica si, al modificar el SP, se debe comprobar que no termine en una dirección anterior a la de inicio de pila (es decir, más baja si la pila crece positivamente y más alta si la pila crece negativamente).
- **Inicio y fin del código:** Indica las direcciones de memoria donde empieza y termina la zona de código.
- **Inicio de la pila:** Indica la dirección de memoria donde empieza la zona de pila.
- **Crecimiento de pila:** Indica la dirección actual de crecimiento de la pila. El crecimiento es positivo cuando crece de direcciones menores a mayores, y negativo cuando sucede al revés (también se usan los términos “pila creciente” y “pila decreciente” de manera intercambiable).
- **Representación numérica:** Indica el formato actual de representación numérica. Las tres posibles representaciones que pueden tomar los valores enteros del sistema son decimal con signo, decimal sin signo y hexadecimal (sin signo). Esta forma de representación solamente se tiene en cuenta cuando los valores se muestran al usuario; la representación interna se mantiene homogénea.

Al ensamblar y cargar en la memoria un nuevo programa, se calculan las direcciones de inicio y fin de las zonas de pila y código mencionadas más arriba. La dirección inicial de la zona de código queda definida como la dirección de menor valor en la que se haya codificado una instrucción, insertado un dato o reservado memoria, al igual que la dirección más alta en la que sucede esto queda definida como la dirección final. Por otra parte, la zona inicial de la pila será asignada, en función de su dirección de crecimiento, a la menor o mayor posición del bloque de memoria vacío más grande disponible, mientras que la posición final cambiará dinámicamente con el valor del SP. Las direcciones de ambas zonas pueden ser modificadas por el usuario.

La zona inicial de pila se calcula en función de dónde esté ubicado el código en la memoria; es decir, si deja un espacio vacío antes de empezar la zona de código, y cómo de grande es dicho espacio en comparación al sector de memoria vacío después de la zona de código. También se tiene en cuenta la dirección de crecimiento de la pila. El criterio a seguir es el siguiente:

- Si el crecimiento es negativo, la dirección inicial de pila será la dirección de mayor valor de la zona de mayor tamaño. Se prioriza la zona anterior al código en caso de igual tamaño que la posterior, para minimizar el riesgo de invasión de zona de código por el SP.
- Si el crecimiento es positivo, la dirección inicial de pila será la dirección de menor valor de la zona de mayor tamaño. Se prioriza la zona posterior al código en caso de igual tamaño por el mismo motivo que en el caso anterior.

El propósito de la existencia de ambas zonas es facilitar la depuración mediante la activación de los parámetros de comprobación de invasión. Estos parámetros permiten que se genere un error si en algún momento el PC entra dentro de la zona de pila o si el SP entra dentro de la zona de código, deteniendo la ejecución.

4. Estructura del código ensamblador

El código fuente es el texto plano que será interpretado por el sistema para su posterior ensamblado en instrucciones ejecutables. Los ficheros de código fuente siguen una estructura sencilla compuesta por una sucesión de líneas. No existe un número límite de caracteres por línea, por lo que la única limitación para la longitud de las etiquetas y los comentarios dependerá del sistema operativo o la memoria disponible.

El fichero debe estar codificado en UTF-8 sin BOM (Byte Order Mark). Si se detecta otra codificación, se instará al usuario a recodificar el fichero o, en su defecto, a eliminar los caracteres fuera del ASCII estándar (como letras con tilde) para tratar de hacerlo compatible². En caso de usar UTF con BOM, es posible que el fichero sea aceptado inicialmente, pero el propio BOM será leído como un carácter no válido para el procesador de código y se generará un error.

Por otro lado, se permite la inserción de tantas líneas en blanco y/o con comentarios entre la declaración de una etiqueta y la instrucción o directiva como se desee, lo que deja una estructura general con el siguiente formato:

```
[[Etiqueta ':' ] [\n [';' Comentario]]* Instrucción|Directiva [ ';' Comentario]
```

Dicho de otro modo:

- La declaración de una etiqueta siempre debe ir seguida de ':', y siempre en la misma línea que la etiqueta.
- La declaración de una instrucción o directiva debe suceder por completo dentro de la misma línea.
- Una etiqueta solamente puede ir seguida de una instrucción o directiva, nunca de otra etiqueta.
- Se ignorará todo texto situado a la derecha de ';', incluido dicho carácter, hasta el final de línea.

Algunos ejemplos correctos e incorrectos se pueden observar, respectivamente, en las figuras 1 y 2:

```
1          ADD #32, /24 ; Esto es un comentario
2      etiq1:  BR \ $40
3      etiq2:
4          ; Otro comentario
5          HALT
6
```

Figura 1: Ejemplos válidos de instrucciones y comentarios.

El formato que comparten las instrucciones es el siguiente:

²Esta solución no garantiza la compatibilidad, y se recomienda recurrir simplemente a guardar de nuevo el fichero usando la codificación UTF-8 sin BOM.

```

1      etiq1 ADD #32, /24 ; ERROR: No se ha usado el separador ':' tras
      la etiqueta.
2
3      etiq2:  etiq3: BR \ $40 ; ERROR: No se permite doble etiquetado
4
5      etiq4:  ADD #32, ; Comentario ; /24 ; ERROR: los comentarios no
      pueden abrirse y cerrarse
6      HALT
7

```

Figura 2: Ejemplos inválidos de instrucciones y comentarios.

Mnemónico [Operando[',' Operando]]

Sin embargo, como se describirá más adelante, las operaciones están segmentadas según su número de operandos, por lo que el formato mencionado es una manera general de establecer su estructura común. En realidad, la estructura es fija para cada tipo, como se puede ver en la tabla 3.

N. Operandos	Estructura
0	Mnemónico
1	Mnemónico Operando
2	Mnemónico Operando ',' Operando

Tabla 3: Estructura de una instrucción según sus operandos

Donde el mnemónico es, esencialmente, el identificador de la instrucción. Como se ha mencionado, existen también las directivas del ensamblador, que se verán en su propia sección más adelante.

Por último, cabe destacar que el lenguaje es *case-sensitive*, esto es, distingue entre mayúsculas y minúsculas. No es lo mismo definir la etiqueta `etiq1` que la etiqueta `ETIQ1`, por ejemplo. Por otro lado, las instrucciones y directivas se aceptan únicamente en un mismo formato de capitalización (todo en mayúscula o todo en minúscula). Por ejemplo:

- `HALT`: Instrucción correcta.
- `halt`: Instrucción correcta.
- `Halt`: Instrucción incorrecta (puede usarse como etiqueta).

5. Modos de direccionamiento

Los modos de direccionamiento indican al simulador dónde y cómo debe buscar y almacenar los datos requeridos y generados por una instrucción. ENS2025 permite un total de seis modos de direccionamiento. Puesto que se representan con 3 bits (lo que da un máximo de 8 modos representables), uno de los modos se desdobra, como se verá a continuación:

- **Direccionamiento inmediato.**

En este modo de direccionamiento, el valor final del operando está explícito en el mismo. En la arquitectura de este sistema, eso significa que se emplean los 16 bits de palabra completos para definir dicho valor en complemento a 2.

Este modo de direccionamiento va indicado por un carácter de almohadilla ('#') precediendo al valor del operando.

Los valores pueden representarse de forma decimal con signo y sin signo, y mediante hexadecimal sin signo. Así, los operandos #100 y #0x64 son equivalentes entre sí, y del mismo modo lo son entre sí #-1, #65535 y #0xFFFF.

Por ejemplo, la instrucción `CMP #23, .R1` compara el valor 23 con el que esté almacenado en el registro R1.

- **Direccionamiento directo a registro.**

En este modo, el operando contiene un mnemónico con el nombre del registro donde se almacena el valor. Dado que el banco de registros del sistema propuesto contiene 16 registros, serán necesarios 4 bits para representar el identificador del registro escogido.

Este modo de direccionamiento va indicado por un punto ('.') precediendo al mnemónico del registro. Al igual que sucede con los mnemónicos de instrucción, se puede representar el mnemónico del registro mediante mayúsculas o minúsculas, siendo equivalentes los operandos `.r0` y `.R0`.

Por ejemplo, la instrucción `INC .R0` incrementa en 1 el valor almacenado en el registro R0.

- **Direccionamiento directo a memoria.**

En este modo de direccionamiento, el operando expresa la dirección absoluta donde se encuentra el valor del mismo. Al igual que sucede con el direccionamiento inmediato, se requiere una palabra completa de 16 bits para expresar este direccionamiento para poder representar todas las posiciones de memoria disponibles.

Este modo de direccionamiento va indicado por un carácter de barra inclinada ('/') precediendo al valor del operando.

Al igual que en el direccionamiento inmediato, los valores pueden representarse de forma decimal con signo y sin signo, y mediante hexadecimal sin signo. Así, los operandos `/100` y `/0x64` son equivalentes entre sí (representando la dirección de memoria 100), y del mismo modo lo son entre sí `/-1`, `/65535` y `/0xFFFF` (representando la dirección de memoria 65535).

Por ejemplo, la instrucción `INC /0x0500` incrementa en 1 el valor almacenado en la posición de memoria 0x0500.

- **Direccionamiento indirecto.**

En este modo de direccionamiento, el operando contiene un mnemónico con el nombre de un registro, y en dicho registro se almacena la dirección donde se encuentra el valor. Al igual que en el modo directo a registro, se necesitan 4 bits para representar el identificador del registro escogido.

Este modo de direccionamiento se representa de manera similar a la del directo a registro (el mnemónico precedido de un punto), pero insertando dicha forma de representación entre corchetes (por ejemplo, `[.R0]`).

Por ejemplo, sea R2 con un valor almacenado de 100. La instrucción `DEC [.R2]` decrementará en 1 el valor almacenado en la posición de memoria 100 (es decir, equivale a la instrucción `DEC /100`).

- **Direccionamiento relativo a registro índice.**

En este modo de direccionamiento, el operando expresa un desplazamiento sobre la dirección marcada por un puntero. El rango de desplazamiento respecto al puntero es de 8 bits en complemento a 2, lo que otorga un rango de desplazamiento de -128 a 127 posiciones de memoria respecto a la posición actual del puntero.

En esta implementación del sistema, los únicos punteros permitidos son los registros de índice IX e IY (por lo tanto, para acortar, es preferible referirse a este direccionamiento como *relativo a IX* o *relativo a IY*). A cada uno de ellos les corresponde un código de direccionamiento distinto, por lo que solamente son necesarios los 8 bits del desplazamiento para representar cada direccionamiento.

Este modo de direccionamiento se representa mediante una combinación del direccionamiento inmediato y el indirecto: el valor numérico va precedido por la almohadilla ('#') y seguido por el registro en cuestión (IX o IY) en notación indirecta (precedido por un punto y entre corchetes).

Al margen de la diferencia de tamaño, se aplican las mismas reglas de interpretación numérica que con los direccionamientos inmediato y directo a memoria: las instrucciones `INC #64[.IX]` y `#0x40[.IX]` incrementan en 1 el valor almacenado en la dirección situada 40 posiciones *después* de la almacenada en IX, mientras que las instrucciones `INC #FF[.IY]`, `INC #-1[.IY]` y `INC #255[.IY]` incrementan en 1 el valor almacenado en la dirección situada 1 posición *antes* de la almacenada en IY.

Por ejemplo, sea IX con un valor almacenado de 100. La instrucción `MOVE #200, #25[.IX]` almacenará el valor 200 en la posición de memoria 125 (es decir, sería lo mismo que la instrucción `MOVE #200, /125`), y la instrucción `MOVE #200, #-10[.IX]` almacenará el valor 200 en la posición de memoria 90.

- **Direccionamiento relativo a contador de programa.**

Similar al relativo a índice, este modo de direccionamiento requiere 8 bits en complemento a 2 para representar un desplazamiento de -128 a 127 posiciones respecto al puntero. La diferencia aquí es que el puntero es el valor del contador de programa *después* de ejecutar la instrucción.

Este modo de direccionamiento se representa mediante el carácter del símbolo de dólar ('\$'), seguido del valor del desplazamiento.

Al igual que en el relativo a índice, en este direccionamiento también se aplican las mismas reglas de representación numérica: las instrucciones `BR $0xFC`, `BR $-4` y `BR $251` son equivalentes.

Por ejemplo, sea la instrucción `BR $20` situada en la dirección 0x0000. Cuando termine de ser leída, el valor del PC antes de ejecutar la instrucción será de 0x0002, ya que es una instrucción que ocupa dos palabras. Por lo tanto, el valor final del PC una vez se ejecute la instrucción será de 0x0022.

Sin embargo, este comportamiento no es universal. Existen ciertas diferencias de comportamiento para algunos modos de direccionamiento en función del tipo de instrucción; concretamente,

variaciones para las instrucciones de gestión de cadenas de caracteres (INSTR y WRSTR) y las instrucciones de salto. Por lo tanto, aunque es en el apartado siguiente donde se clasifican con propiedad los tipos de instrucciones y se define la función a realizar por cada una, así como los modos de direccionamiento que acepta, es necesario profundizar en esta situación.

En primer lugar, se detalla el comportamiento de los modos de direccionamiento para las instrucciones de salto:

- **Direccionamiento directo a memoria.**

Se trata como un direccionamiento inmediato, ya que este último no se emplea en este tipo de instrucciones. Se usa el valor literal de la dirección, no el contenido de la misma. Por ejemplo, la instrucción `BR /1000` establece para el PC el valor 1000, no el valor contenido en la dirección 1000. Es decir, equivale a `MOVE #1000, .PC`, no a `MOVE /1000, .PC`.

- **Direccionamiento indirecto.**

El direccionamiento indirecto funciona de manera idéntica a la estándar.

- **Direccionamiento relativo a contador de programa.**

El direccionamiento relativo a contador de programa funciona de manera idéntica a la estándar.

Finalmente, se detalla el comportamiento de los modos de direccionamiento empleados por INSTR y WRSTR para determinar la posición donde se va a comenzar a leer o escribir la cadena:

- **Direccionamiento directo a memoria.**

Funciona igual que con las instrucciones de salto: se interpreta el operando como si fuera un direccionamiento inmediato. Es decir, `WRSTR /100` equivaldría a `WRSTR #100` si se tuviera que definir su comportamiento en base a las reglas generales de direccionamiento.

- **Direccionamiento indirecto.**

Se trata como un direccionamiento a registro. Es decir, en vez de usar el contenido del registro como dirección de memoria que apunta al primer carácter de la cadena, el valor contenido en el registro *es* la dirección del primer carácter de la cadena. Por lo tanto, `WRSTR [.R0]` equivaldría a `WRSTR .R0` si su comportamiento se tuviera que definir usando las reglas para los casos generales.

- **Direccionamiento relativo a registro índice.**

En este caso, el desplazamiento sobre el registro IX o IY apunta al primer carácter de la cadena, no al valor de la dirección de dicho carácter. Así, para un valor de 100 almacenado en IX, si la instrucción `INC #25[.IX]` equivale a `INC /125`, bajo el comportamiento general la instrucción `WRSTR #25[.IX]` tal y como funciona actualmente equivaldría en realidad a `WRSTR #125`.

Los identificadores de los modos de direccionamiento pueden consultarse en la tabla 4.

Código	Bits	Modo de direccionamiento	Ancho de representación
0	000	Sin operandos	N/A
1	001	Inmediato	16 bits
2	010	Registro	4 bits
3	011	Memoria	16 bits
4	100	Indirecto	4 bits
5	101	Relativo a IX	8 bits
6	110	Relativo a IY	8 bits
7	111	Relativo a PC	8 bits

Tabla 4: Tabla de códigos de direccionamiento.

6. Juego de instrucciones

En esta sección se listan todas las instrucciones soportadas por ENS2025. Dichas instrucciones están agrupadas de manera conceptual en los siguientes conjuntos:

- **Instrucciones de transferencia de datos:** MOVE, PUSH, POP.
- **Instrucciones aritméticas:** ADD, SUB, MUL, DIV, MOD, INC, DEC, NEG.
- **Instrucciones de comparación:** CMP.
- **Instrucciones lógicas:** AND, OR, XOR, NOT.
- **Instrucciones de salto (bifurcación):** BR, BZ, BNZ, BP, BN, BV, BNV, BC, BNC, BE, BO.
- **Instrucciones de manejo de subrutinas:** CALL, RET.
- **Instrucciones de entrada/salida:** INCHAR, ININT, INSTR, WRCHAR, WRINT, WRSTR.
- **Otras instrucciones:** NOP, HALT.

A continuación, se listan individualmente, indicando su mnemónico, su función, el número de operandos que contiene y los modos de direccionamiento que admite para cada uno, y su comportamiento:

- **Instrucción NOP:**
 - **Descripción:** No operación.
 - **Número de operandos:** 0.
 - **Direccionamientos:** Ninguno.
 - **Comportamiento:** Ninguno.
 - **Modifica biestables:** No.
- **Instrucción HALT:**

- **Descripción:** Detener ejecución.
- **Número de operandos:** 0.
- **Direccionamientos:** Ninguno.
- **Comportamiento:** Activa el biestable H.
- **Modifica biestables:** No (el resto de biestables mantienen su último valor, no se reinician).

■ **Instrucción MOVE:**

- **Descripción:** Copia de valores entre operandos.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Copia el contenido de OP1 en OP2.
- **Modifica biestables:** No

■ **Instrucción PUSH:**

- **Descripción:** Inserción en pila.
- **Número de operandos:** 1.
- **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
- **Comportamiento:** Almacena el valor contenido en el operador en la dirección de memoria apuntada por el puntero de pila (SP) y luego desplaza el puntero de pila una posición en la dirección de crecimiento de la misma.
- **Modifica biestables:** No.

■ **Instrucción POP:**

- **Descripción:** Extracción de la pila.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo.
- **Comportamiento:** Desplaza el puntero de pila (SP) una posición en dirección opuesta al crecimiento de la misma y luego almacena el valor contenido en la nueva dirección apuntada por el SP en el operando.
- **Modifica biestables:** No.

■ **Instrucción ADD:**

- **Descripción:** Suma de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **Comportamiento:** Suma los contenidos de OP1 y OP2 y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí

■ **Instrucción SUB:**

- **Descripción:** Resta de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Resta los contenidos de OP1 y OP2 (OP1-OP2) y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí.

■ **Instrucción MUL:**

- **Descripción:** Multiplicación de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Multiplica los contenidos de OP1 y OP2 y almacena el resultado en el registro acumulador (A).
- **Modifica biestables:** Sí.

■ **Instrucción DIV:**

- **Descripción:** División de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Divide OP1 entre OP2 y almacena el resultado en el registro acumulador (A). Si OP2 tiene valor 0, se produce un error.
- **Modifica biestables:** Sí.

■ **Instrucción MOD:**

- **Descripción:** Resto de la división de enteros.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **Comportamiento:** Resta los contenidos de OP1 y OP2 y almacena el resto de dicha división en el registro acumulador (A). Si OP2 tiene valor 0, se produce un error.
- **Modifica biestables:** Sí.

■ **Instrucción INC:**

- **Descripción:** Incremento unitario.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Incrementa en 1 el contenido de OP1.
- **Modifica biestables:** Sí.

■ **Instrucción DEC:**

- **Descripción:** Decremento unitario.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Decrementa en 1 el contenido de OP1.
- **Modifica biestables:** Sí.

■ **Instrucción NEG:**

- **Descripción:** Cambio de signo.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** Cambia de signo el operando mediante la operación $0-OP1$.
- **Modifica biestables:** Sí.

■ **Instrucción CMP:**

- **Descripción:** Comparación.
- **Número de operandos:** 2
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
- **Comportamiento:** El mismo que en la instrucción SUB, pero sin almacenar el resultado de la operación en ningún sitio.
- **Modifica biestables:** Sí.

■ **Instrucción AND:**

- **Descripción:** AND lógico.
- **Número de operandos:** 2.
- **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.

- **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Efectúa la operación lógica AND entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si los bits de ambos operandos en esa misma posición tienen valor 1. Almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** No.
- **Instrucción OR:**
- **Descripción:** OR lógico.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Efectúa la operación lógica OR entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si el bit de uno o ambos operandos en esa misma posición tienen valor 1. Almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** No.
- **Instrucción XOR:**
- **Descripción:** XOR lógico.
 - **Número de operandos:** 2.
 - **Direccionamientos:**
 - **OP1:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **OP2:** Inmediato, registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Efectúa la operación lógica XOR entre ambos operandos bit a bit. El bit de cada posición del resultado tendrá valor 1 si los bits de ambos operandos en esa misma posición tienen distintos valores (uno 0 y el otro 1). Almacena el resultado en el registro acumulador (A).
 - **Modifica biestables:** No.
- **Instrucción NOT:**
- **Descripción:** Negación lógica.
 - **Número de operandos:** 1.
 - **Direccionamientos:** Registro, memoria, indirecto, relativo IX/IY.
 - **Comportamiento:** Invierte bit a bit el valor del operando (cambia los 0 por 1 y viceversa).
 - **Modifica biestables:** No.
- **Instrucción BR:**
- **Descripción:** Bifurcación incondicional.

- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 .
- **Modifica biestables:** No.

■ **Instrucción BZ:**

- **Descripción:** Bifurcación si $Z=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable Z es 1.
- **Modifica biestables:** No.

■ **Instrucción BNZ:**

- **Descripción:** Bifurcación si $Z=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable Z es 0.
- **Modifica biestables:** No.

■ **Instrucción BP:**

- **Descripción:** Bifurcación si $S=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable S es 0.
- **Modifica biestables:** No.

■ **Instrucción BN:**

- **Descripción:** Bifurcación si $S=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable S es 1.
- **Modifica biestables:** No.

■ **Instrucción BV:**

- **Descripción:** Bifurcación si $V=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable V es 1.

- **Modifica biestables:** No.

■ **Instrucción BNV:**

- **Descripción:** Bifurcación si $V=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable V es 0.
- **Modifica biestables:** No.

■ **Instrucción BC:**

- **Descripción:** Bifurcación si $C=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable C es 1.
- **Modifica biestables:** No.

■ **Instrucción BNC:**

- **Descripción:** Bifurcación si $C=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable C es 0.
- **Modifica biestables:** No.

■ **Instrucción BE:**

- **Descripción:** Bifurcación si $P=0$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable P es 0.
- **Modifica biestables:** No.

■ **Instrucción B0:**

- **Descripción:** Bifurcación si $P=1$.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Carga el PC con el valor de OP1 si el valor del biestable P es 1.

- **Modifica biestables:** No.

■ **Instrucción CALL:**

- **Descripción:** Llamada a subrutina.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, relativo a contador de programa, indirecto.
- **Comportamiento:** Almacena el valor actual del PC en la dirección de memoria apuntada por el SP, desplaza el SP una posición en la dirección de crecimiento de la pila y carga en el PC el valor almacenado en el operando (es decir, **CALL <OP>** equivale a **PUSH .PC** seguido de **BR <OP>**, siendo <OP> un operando genérico con direccionamiento permitido).
- **Modifica biestables:** No.

■ **Instrucción RET:**

- **Descripción:** Retorno de subrutina.
- **Número de operandos:** 0.
- **Direccionamientos:** Ninguno.
- **Comportamiento:** Desplaza el SP una posición en la dirección opuesta al crecimiento de la pila y carga en el PC el valor almacenado en la nueva dirección del SP (es decir, **RET** equivale a **POP .PC**)
- **Modifica biestables:** No.

■ **Instrucción INCHAR:**

- **Descripción:** Lectura de carácter.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo.
- **Comportamiento:** Lee un carácter de la consola y lo codifica en ASCII, dejando a cero los 8 bits superiores de la palabra de 16 bits y almacenando el carácter leído en el operando.
- **Modifica biestables:** No.

■ **Instrucción ININT:**

- **Descripción:** Lectura de entero.
- **Número de operandos:** 1.
- **Direccionamientos:** Registro, memoria, indirecto, relativo.
- **Comportamiento:** Lee un entero de la consola. Si la entrada no puede ser convertida en un entero (formato incorrecto, valor fuera de rango, entrada vacía), se considerará que se ha introducido un 0. Los enteros se pueden escribir en formato decimal (con y sin signo) y hexadecimal (sin signo y precedido por "0x").
- **Modifica biestables:** No.

■ **Instrucción INSTR:**

- **Descripción:** Lectura de cadena.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, indirecto, relativo.
- **Comportamiento:** Lee una cadena de caracteres de la consola y la almacena en posiciones consecutivas de memoria a partir de la dirección indicada por el operando, acabada con el carácter ‘\0’. No se comprueba previamente si hay espacio para almacenar la cadena, por lo que se produce un error si se sobrepasa el límite superior de la memoria.
- **Modifica biestables:** No.

■ **Instrucción WRCHAR:**

- **Descripción:** Escritura de carácter.
- **Número de operandos:** 1.
- **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
- **Comportamiento:** Escribe en la consola el carácter cuyo valor ASCII se corresponde con los 8 bits inferiores del valor del operando.
- **Modifica biestables:** No.

■ **Instrucción WRINT:**

- **Descripción:** Escritura de entero.
- **Número de operandos:** 1.
- **Direccionamientos:** Inmediato, registro, memoria, indirecto, relativo.
- **Comportamiento:** Escribe en la consola el valor del operando. El formato de representación numérica (hexadecimal, numeral con/sin signo) dependerá de la configuración de ese parámetro en el sistema.
- **Modifica biestables:** No.

■ **Instrucción WRSTR:**

- **Descripción:** Escritura de cadena.
- **Número de operandos:** 1.
- **Direccionamientos:** Memoria, indirecto, relativo.
- **Comportamiento:** Escribe en la consola una cadena de caracteres, que estará almacenada en memoria a partir de la dirección indicada por el operando. Se escribirán caracteres hasta que se llegue al carácter ‘\0’. Por tanto, puede ocurrir que durante este proceso se genere un error si no se encuentra el carácter de fin de cadena antes de llegar al límite superior de la memoria.
- **Modifica biestables:** No.

7. Etiquetas

Las etiquetas son mnemónicos definidos por el usuario que sirven como nombres simbólicos. Comienzan por un carácter alfabético, seguido o no de cualquier combinación de caracteres alfanuméricos y de barra baja ('_'). Solamente pueden ser definidos precediendo a una instrucción o directiva, y siempre irán finalizados por un carácter de dos puntos (':') a modo de separador (es decir, no es parte del mnemónico como tal). Toman el valor que tenga el puntero de escritura cuando son declaradas, antes del procesamiento de una instrucción o la ejecución de una directiva, salvo si se trata de la directiva EQU, en cuyo caso toman como valor el resultado de la expresión aritmética asociada a la directiva.

Las etiquetas pueden usarse como sustitución de cualquier literal entero que forme parte de un operando, como por ejemplo WRCHAR #VALOR, BR \$ini, WRSTR /cadena o ADD #BASE, #DESP[.IX]. La etiqueta se sustituye por su valor pertinente durante el ensamblado.

8. Directivas del ensamblador

Las directivas del ensamblador son órdenes dirigidas al programa ensamblador con el fin de organizar el proceso de ensamblado; pero no forman parte del código y, por tanto, no se codifican en lenguaje máquina para su posterior simulación.

Las directivas definidas para este sistema (así como sus comportamientos), son las siguientes:

- **Directiva ORG:** Tiene como argumento una expresión aritmética. El resultado de dicha expresión determinará la dirección de memoria a la que se moverá el puntero de escritura. **IMPORTANTE:** No se realiza ninguna comprobación implícita de sobrescritura. Por ejemplo, en el código presentado en la figura 3, la instrucción ADD invade el código escrito más arriba. Si no se incluye directiva ORG en el código, éste se ensamblará a partir de la primera dirección (0).

```
1      ORG 4
2      INC /23
3      HALT
4
5      ORG 0
6      BR /599
7      PUSH #33
8      ADD #200, [.R3]
9
```

Figura 3: Ejemplo de directiva ORG provocando sobrescritura de código.

- **Directiva EQU:** Tiene como argumento una expresión aritmética. Asigna el resultado de dicha expresión a la etiqueta asociada. Si no se indica ninguna etiqueta, la expresión será evaluada igualmente, pero no se asignará su resultado a ningún elemento.

- **Directiva END:** No tiene argumentos. Señala el final del código fuente. No es obligatoria, ya que el proceso de ensamblado finaliza sin errores igualmente una vez que se termina de leer el fichero de código. El ensamblador ignorará todo lo que haya escrito a partir de esta directiva.
- **Directiva RES:** Tiene como argumento una expresión aritmética. El resultado de dicha expresión determinará cuántas posiciones de memoria se reservan (sin inicializar su valor). En caso de ir precedida de una etiqueta, ésta indicará la primera posición reservada.
- **Directiva DATA:** Tiene como argumento una sucesión tan larga como desee el usuario (siempre que la memoria lo permita) de valores numéricos y cadenas de caracteres (que deberán ir entrecomilladas) separados por comas. Las cadenas se almacenarán en memoria indicando su final mediante el carácter ‘\0’ como añadido final. Pueden estar compuestas de cualquier carácter imprimible, y pueden contener los caracteres especiales ‘\0’, ‘\n’ y ‘\t’, así como las propias comillas y la barra invertida usada como escapador de caracteres, siempre que vayan ambos precedidos de otra barra invertida (‘\'’ y ‘\\’).

9. Detección de errores en el código fuente

Además de los errores en tiempo de ejecución mencionados, pueden producirse errores durante la lectura de código. Estos errores aportan información sobre su categoría, su causa y, si procede, la línea en la que se han producido. Por ejemplo, si se intenta ensamblar este código:

```
ADD #-2 #4
```

Se generará este error:

Error en línea 1: Se esperaba el carácter ‘,’ tras el primer operando.

10. Interfaz de línea de comandos

La interfaz de línea de comandos es una aplicación sencilla que utiliza la consola del sistema como medio para su ejecución y comunicación con el usuario. Consiste en un *prompt* que el usuario utiliza para introducir diversos comandos. Estos comandos serán interpretados por el módulo, que llamará a los métodos pertinentes de la unidad de control para cumplir con el cometido esperado de cada uno de ellos.

10.1. Inicio del programa y parámetros de invocación

Al estar compilado en un objeto JAR, la aplicación se inicia introduciendo en la línea de comandos la siguiente invocación:

```
java -jar ens2025.jar [parámetros]
```

Si se invoca sin ningún parámetro, la secuencia de inicio es la siguiente:

1. Se intenta cargar el fichero de configuración.
2. Si no se encuentra, se carga la configuración estándar.
3. Se muestra el estado de la configuración del sistema.
4. Se muestra la ayuda simplificada de los comandos.
5. Se muestra el *prompt* y se devuelve el control al usuario.

Por otro lado, la aplicación puede ser invocada mediante los siguientes parámetros:

- **-a <nombre>**: Inicia la ejecución cargando y ensamblando un fichero de código ensamblador.
- **-m <nombre>**: Inicia la ejecución cargando directamente un fichero binario de imagen de memoria.
- **-rec**: Inicia la ejecución cargando el fichero de recuperación generado al interrumpir la ejecución del programa mientras el proceso de simulación está activo.
- **-r <nombre>**: Carga un fichero ensamblador, lo ensambla, ejecuta el código resultante y finaliza el programa.
- **-rm <nombre>**: Carga una imagen de memoria, la ejecuta automáticamente y finaliza el programa.
- **-d <nombre>**: Si se ha seleccionado un argumento de ejecución (**-r** o **-rm**), este parámetro especifica un nombre de fichero binario de imagen de memoria en el que se volcará el estado del sistema al finalizar la simulación antes de cerrar el programa.
- **-s <nombre>**: Permite cargar un fichero de configuración específico para esta ejecución del programa. La configuración generada por dicho fichero no será permanente: al cerrar el programa e invocarlo sin este parámetro, se cargará con normalidad el fichero de configuración estándar. La configuración durante la ejecución del programa con este parámetro es temporal, y los cambios realizados en la misma no se guardarán ni en el fichero cargado mediante parámetro ni en el fichero estándar.

La finalidad de dichos parámetros es otorgar flexibilidad y fluidez a la ejecución, en especial la ejecución por lotes. De esta manera, tanto los alumnos como el equipo docente pueden hacer comprobaciones de ejecución secuenciales en ciclos de menor duración. La opción de cargar un fichero de configuración permite, además, hacer comprobaciones de un mismo fichero de código con distintas configuraciones (por ejemplo, una ejecución normal sin guardas, otra con las guardas de invasión de punteros y desbordamiento de pila activas, otra con la pila creciendo en la dirección opuesta...) sin necesidad de andar cambiando el fichero para cada ejecución.

Los parámetros pueden invocarse en cualquier orden, pero atendiendo a ciertas restricciones:

- No se puede usar dos veces el mismo parámetro.
- Solamente se puede usar un único parámetro de carga o de ejecución. Es decir, `-a`, `-m`, `-rec`, `-r` y `-rm` son incompatibles entre sí.
- Solamente se puede usar el parámetro `-d` (volcado) si se usa un parámetro de ejecución (`-r` o `-rm`) en la misma invocación.
- El parámetro `-s` es totalmente independiente y no tiene más restricción que la de no ser duplicado.

Por ejemplo, las siguientes invocaciones son correctas:

```
java -jar ens2025.jar -a "fichero de prueba.ens"
```

```
java -jar ens2025.jar -r c:/Users/Me/Documents/test18.ens
```

```
java -jar ens2025.jar -r c:/Users/Me/Documents/test18.ens -d volcado.mem
```

Mientras que las siguientes invocaciones no lo son:

```
java -jar ens2025.jar -a test18.ens -a test19.ens
```

```
java -jar ens2025.jar -a test18.ens -d volcado.mem
```

```
java -jar ens2025.jar -a test18.ens -r test19.ens
```

10.2. Comandos

Los comandos que acepta la aplicación pueden ir, en algunos casos, acompañados de parámetros, tanto <obligatorios> como [opcionales]. La lista completa de comandos, con sus posibles parámetros y funcionalidades, es la siguiente:

- **ENS <f>**: Ensambla el fichero de nombre ‘f’ y carga en la memoria el contenido de las instrucciones ensambladas.
- **DES <dir><num>**: Desensambla ‘num’ instrucciones a partir de la dirección de memoria ‘dir’. Un ejemplo de instrucción desensamblada es el siguiente:

```
[ ] ADD /23, #3599
```

Donde [] indica que esa instrucción no contiene un punto de ruptura en ninguna de las palabras que la forman. De tenerlo, se mostraría con un asterisco dentro ([*]).

- **RUN**: Simula el código con normalidad.
- **PASO**: Muestra la instrucción apuntada por el PC y la ejecuta.
- **DEBUG**: Simula el código hasta encontrar un punto de ruptura.

- **BP <dir>[val]**: Si se invoca sin segundo parámetro, añade un punto de ruptura en la dirección ‘dir’ si no había uno, y lo elimina si ya existía. Si se invoca con segundo argumento, fuerza ese estado. Los valores pueden ser:

- **SI**: Introduce el punto de ruptura.
- **NO**: Elimina el punto de ruptura.

- **LEERMEM <dir><num>**: Muestra ‘num’ posiciones de memoria a partir de la dirección de memoria ‘dir’. Si el número de posiciones a mostrar es mayor que el número de posiciones restantes hasta el final de la memoria, se mostrarán solamente las posiciones disponibles. Un ejemplo de posición de memoria es el siguiente:

[][][] 0x0F38: 0xCB9F

Donde el primer [], al igual que en el desensamblado, indica si hay un punto de ruptura en esa posición, y se marca con un asterisco ([*]); el segundo [] indica la pertenencia a la zona de código y se marca con una C si se cumple ([C]), y el tercer [] indica la pertenencia a la zona de pila, marcándose con una P si pertenece a dicha zona ([P]), o con un > si el SP apunta justo a esa dirección ([>]).

- **ESCMEM <dir><val>**: Escribe el valor ‘val’ en la dirección de memoria ‘dir’.
- **RESMEM**: Restablece a 0 el valor almacenado en todas las posiciones de memoria.
- **LEERPILA <num>**: Muestra ‘num’ posiciones de memoria a partir de la dirección de memoria del puntero de pila, empezando por la cima. Si el número de posiciones a mostrar es mayor que el número de elementos restantes hasta el final de la pila, o si se produce desbordamiento, se mostrarán solamente las posiciones disponibles.
- **CARGAR <f>**: Carga el estado del procesador (memoria, registros, zonas de código y pila) contenida en el fichero de nombre ‘f’. Compatible con ficheros de memoria generados por ENS2001, pero solamente contienen la memoria.
- **GUARDAR <f>**: Guarda el estado actual del procesador (memoria, registros, zonas de código y pila) en el fichero de nombre ‘f’. Estos ficheros pueden ser cargados por ENS2001, pero solamente cargará el estado de la memoria.
- **BANCO**: Imprime por pantalla los valores de todos los registros.
- **REGESC <reg><val>**: Inserta el valor ‘val’ en el registro identificado por ‘reg’.
- **H [com]**: Si se invoca sin parámetros, muestra la ayuda simplificada de todos los comandos. Si se invoca usando el comando ‘com’ como parámetro, muestra la ayuda avanzada para ese comando específico.
- **CONF**: Imprime por pantalla los valores actuales de configuración del sistema.
- **INVPILA [val]**: Modifica la comprobación de invasión de la zona de pila por parte del PC. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.

- **INVCODE [val]**: Modifica la comprobación de invasión de la zona de código por parte del SP. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **DIRPILA [val]**: Modifica la dirección hacia la que se mueve el SP cuando se introduce un elemento en la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **POS**: Crecimiento positivo (“hacia abajo”).
 - **NEG**: Crecimiento negativo (“hacia arriba”).
- **OVERFLOW [val]**: Modifica la comprobación de desbordamiento de la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **UNDERFLOW [val]**: Modifica la comprobación de subdesbordamiento de la pila. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **SETINIC <dir>**: Cambia el valor de la dirección inicial de la zona de código por la suministrada en ‘dir’.
- **SETFINC <dir>**: Cambia el valor de la dirección final de la zona de código por la suministrada en ‘dir’.
- **SETINIP <dir>**: Cambia el valor de la dirección inicial de la zona de pila por la suministrada en ‘dir’.
- **CONFREG [val]**: Modifica la comprobación de reinicio del banco de registros al recibir una orden de simulación mientras el biestable H está en alto. Si se invoca sin parámetros, establece como nuevo valor el opuesto al actual. Si se invoca con un parámetro, fuerza ese estado. Los valores pueden ser:
 - **SI**: Activa la comprobación.
 - **NO**: Desactiva la comprobación.
- **BASE <val>**: Modifica la base numérica en la que se representan los valores. Algunos elementos, como las direcciones de memoria, mostrarán su valor decimal sin signo incluso si la representación es decimal con signo. Los valores pueden ser:
 - **HEX**: Hexadecimal.
 - **DEC**: Decimal con signo.

- USDEC: Decimal sin signo.
- X: Guarda el estado actual de la configuración del sistema en el fichero pertinente y termina la ejecución.

Si durante la ejecución se genera un bucle infinito, se deberá pulsar las teclas CONTROL+C para detenerlo. Si bien esto termina el programa forzosamente, antes se generará un volcado del estado del simulador en un fichero llamado **recovery.dump** que, como ya se ha dicho, se puede cargar invocando el parámetro **-rec** al iniciar el programa.

11. Interfaz Gráfica de Usuario

La interfaz gráfica está pensada para facilitar al usuario el acceso a las mismas funcionalidades que la versión por línea de comandos, en un entorno donde todos los elementos estén visualmente presentes, facilitando su uso. La aplicación está dividida en cinco elementos principales, como se puede ver en la figura 4: la barra de herramientas (1), el banco de registros y biestables (2), el visor de memoria (3), el visor de código(4) y las zonas de texto para los mensajes del sistema y la consola E/S de la simulación (5).

Antes de profundizar en los distintos elementos de la GUI, es conveniente explicar brevemente cómo lanzar esta versión de ENS2025.

11.1. Ejecución de la GUI

El precio a pagar por un ejecutable portable de Java que no requiera pasar por un proceso de instalación y desinstalación es la necesidad de mantener un directorio con los elementos necesarios para la ejecución. Dicho directorio se suministra en forma de una carpeta llamada ENS2025-GUI. En función del sistema operativo desde el que se trabaje, el sistema de ejecución varía ligeramente:

- **Windows.**

Junto a la carpeta ENS2025-GUI se suministra un fichero .bat, que consiste simplemente en el comando `./ENS2025-GUI/bin/ens2025.bat`. Para lanzar ENS2025, basta con hacer doble clic en este fichero. El .bat interno es el verdadero lanzador de la aplicación, que no puede ser movido de la carpeta.

- **Linux.**

De manera similar, se suministra junto a la carpeta ENS2025-GUI un fichero que contiene el comando `./ENS2025-GUI/bin/ens2025`. Para lanzar ENS2025, se puede ejecutar este fichero desde la consola, o hacer clic derecho y seleccionar "Ejecutar como programa".

- **MacOS.**

Se suministra junto a la carpeta ENS2025-GUI un alias que dirige al script de ejecución. Para lanzar ENS2025, basta con hacer doble clic en este alias. También se puede hacer doble clic en el script situado en `ENS2025-GUI/bin/ens2025`.

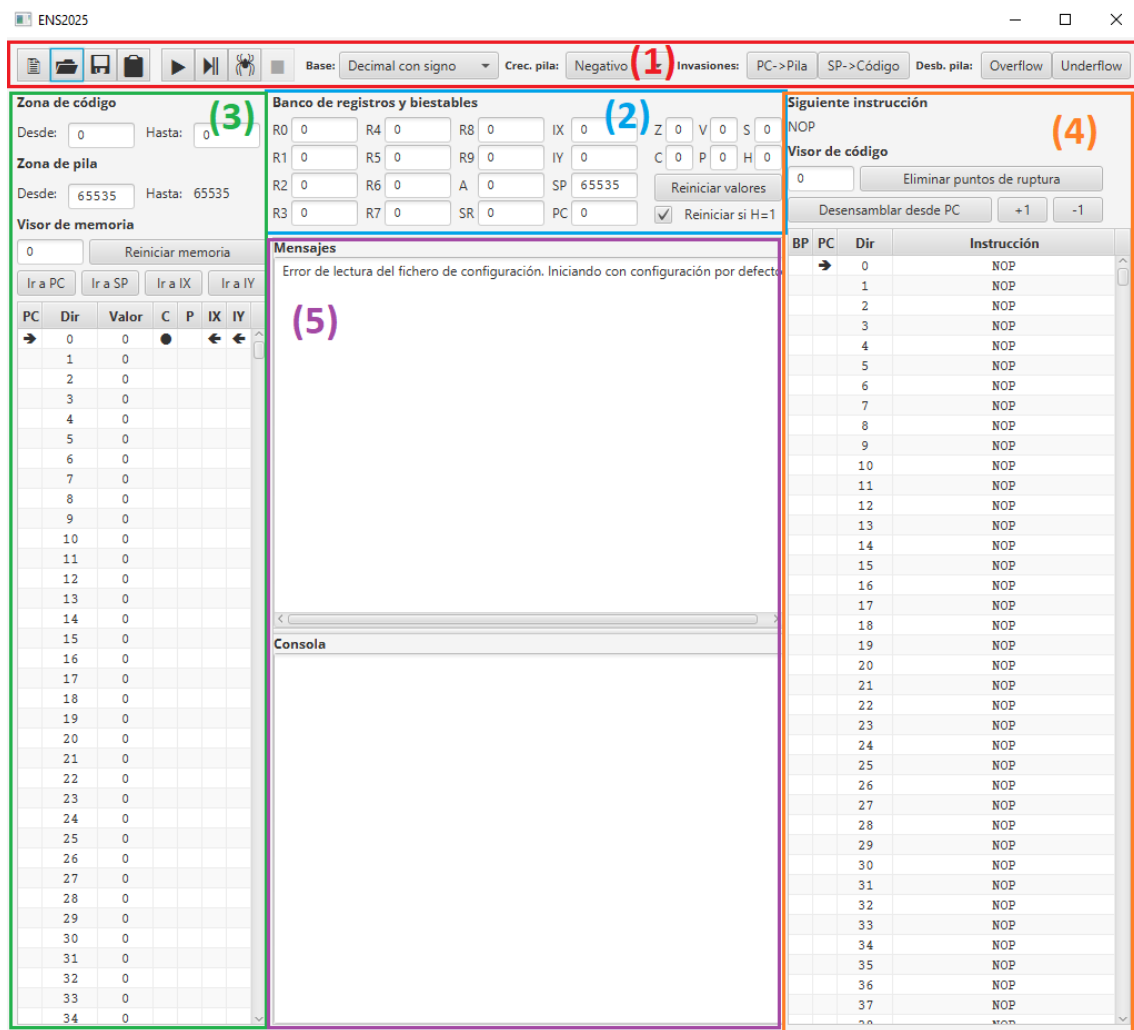


Figura 4: Vista general de la ventana de interfaz gráfica.

11.2. La barra de herramientas

La parte superior de la GUI incluye todos los elementos de configuración y ejecución en una misma barra de herramientas, visualmente dividida en tres secciones:

- **Gestión de ficheros:** Contiene los elementos dedicados a la carga y guardado de ficheros, como se puede ver en la figura 5a. Los elementos son, en orden, los siguientes:
 - **Botón de ensamblado de fichero:** Abre una ventana de exploración de archivos, mostrando por defecto solamente los ficheros con extensión `.ens`, con la finalidad de seleccionar un fichero de código fuente ensamblador. El fichero seleccionado será ensamblado y cargado en la memoria.
 - **Botón de carga de imagen:** Abre una ventana de exploración de archivos, mostrando por defecto solamente los ficheros con extensión `.mem`, con la finalidad de seleccionar un fichero binario de imagen de memoria. El fichero seleccionado se cargará en la memoria.
 - **Botón de guardado de imagen:** Abre una ventana de exploración para designar la localización y nombre del fichero binario a escribir (o sobrescribir) con la imagen

de memoria de sistema.

- **Botón de guardado de mensajes:** Abre una ventana de exploración para designar la localización y nombre del fichero de texto a escribir (o sobrescribir) con un volcado del área de texto de mensajes, seguido de un volcado del área de texto de la consola de entrada/salida.
- **Gestión de simulación:** Contiene los elementos dedicados al inicio y parada del simulador, como se puede ver en la figura 5b. Los elementos son, en orden, los siguientes:
 - **Botón de ejecución estándar:** Inicia la ejecución estándar del simulador (de manera continua y sin detenerse ante puntos de ruptura).
 - **Botón de ejecución paso a paso:** Ejecuta una sola instrucción.
 - **Botón de depuración:** Inicia la simulación con la condición de detenerse ante un punto de ruptura. Si la depuración se inicia mientras el PC apunta a una instrucción que contiene un punto de ruptura, dicho punto de ruptura se ignora para evitar que haya que desactivarlo para poder iniciar la simulación.
 - **Botón de parada:** Normalmente deshabilitado, se habilita cuando uno de los modos de ejecución está activo, dando la opción de detenerlo prematuramente.
- **Gestión de configuración:** Contiene los elementos dedicados a la visualización y modificación de los valores de configuración del sistema, como se puede ver en la figura 5c. Los elementos son, en orden, los siguientes:
 - **Desplegable de base numérica:** Permite al usuario modificar el formato de representación de los valores.
 - **Desplegable del crecimiento de pila:** Permite al usuario modificar la dirección de crecimiento de la pila.
 - **Botón de alternancia de invasión de pila:** Permite al usuario activar o desactivar la comprobación de invasión de la zona de pila por el PC. En la figura 5c se puede ver que este botón está activado, por lo que la comprobación de invasión de pila también lo está.
 - **Botón de alternancia de invasión de código:** Permite al usuario activar o desactivar la comprobación de invasión de la zona de código por el SP. En la figura 5c se puede ver que este botón está desactivado, por lo que la comprobación de invasión de código tampoco lo está.
 - **Botón de alternancia de comprobación de desbordamiento:** Permite al usuario activar o desactivar la comprobación de desbordamiento de la pila. En la figura 5c se puede ver que este botón está desactivado, por lo que la comprobación de desbordamiento tampoco lo está.
 - **Botón de alternancia de comprobación de subdesbordamiento:** Permite al usuario activar o desactivar la comprobación de subdesbordamiento de la pila. En la figura 5c se puede ver que este botón está activado, por lo que la comprobación de subdesbordamiento también lo está.



(a) Herramientas de archivos.



(b) Herramientas de simulación.



(c) Herramientas de configuración.

Figura 5: Divisiones de la barra de herramientas.

11.3. El banco de registros y biestables

Este componente, que se puede ver con más detalle en la figura 6 contiene los siguientes elementos:

- Campos de texto para visualizar y editar los 16 registros.
- Campos de texto para visualizar y editar los 6 biestables.
- Botón para reiniciar los valores de todos los campos.
- Opción para reiniciar los valores del banco automáticamente si se inicia una simulación mientras el biestable H está en alto.

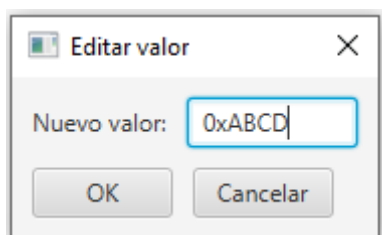
Banco de registros y biestables

R0	0	R4	0	R8	0	IX	0	Z	0	V	0	S	0
R1	0	R5	0	R9	0	IV	0	C	0	P	0	H	0
R2	0	R6	0	A	0	SP	65535	Reiniciar valores ☒ Reiniciar si H=1					
R3	0	R7	0	SR	0	PC	0						

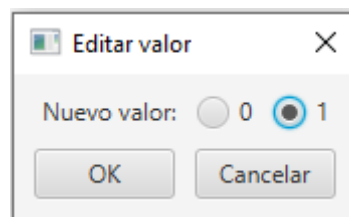
Figura 6: Banco de registros y biestables. El registro A tiene el foco.

Los campos no se pueden editar directamente, sino que requieren hacer doble clic sobre ellos, o pulsar la tecla ENTER mientras el campo deseado tiene el foco (borde azul). En ambos casos, se abre una pequeña ventana de diálogo para elegir el nuevo valor: para los campos de los registros, solicita introducir el valor nuevo (respetando siempre los formatos permitidos y rangos de valores), mientras que para los campos de biestables se pide seleccionar un botón radial para poner el valor a 0 o 1. Ambas variaciones se pueden ver en la figura 7. El funcionamiento de los campos de texto de los registros se aplica a todos los campos de texto similares de la aplicación. Mientras el cuadro de diálogo de modificación de un campo está activo, no se puede interactuar con el resto de la aplicación.

El botón de reiniciar valores, como su nombre indica, restablece a 0 todos los registros y biestables, con excepción del asociado con el SP, que se restablecerá al valor de la dirección inicial de la pila. El **CheckBox** activa o desactiva la comprobación del valor del biestable H antes de cada ejecución.



(a) Edición de campos de valor generales.



(b) Edición de biestables.

Figura 7: Cuadros de diálogo de edición de valores.

11.4. El visor de memoria

Este componente, que se puede ver con más detalle en la figura 8, se encarga de la gestión de la memoria por parte de la aplicación. Permite explorar la memoria y modificar su contenido, así como visualizar y modificar las zonas de código y pila. Por último, mantiene información visual sobre las posiciones de los distintos punteros del sistema y permite saltar hacia ellos para ver el estado de la memoria en sus inmediaciones. Agrupa los siguientes elementos:

- Módulo de visualización y edición de las zonas de código y pila.
- Barra de herramientas del visor.
- El propio visor de memoria.

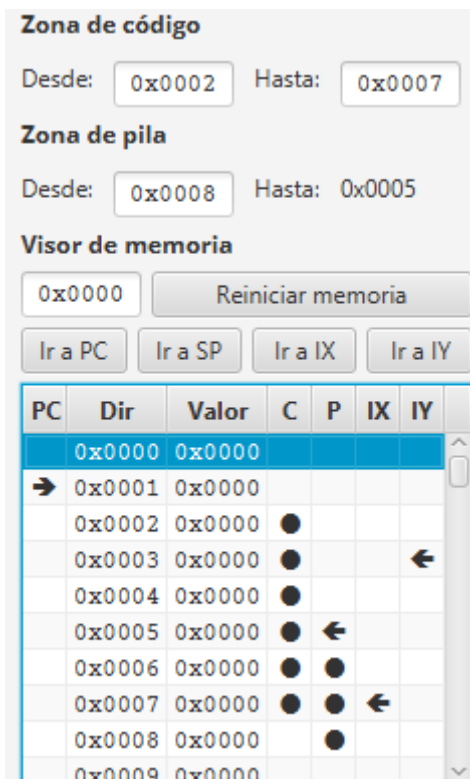


Figura 8: Visor de memoria con su barra de herramientas.

El editor de zona de código contiene dos campos donde se reflejan automáticamente los valores de las direcciones inicial y final del código desde el momento en que se carga y ensambla un

fichero. Al igual que con los campos de valores de los registros, hacer doble clic o pulsar ENTER con el foco en uno de estos campos permite su modificación, cambiando el tamaño designado de la zona de código. Por otro lado, el editor de la zona de pila, si bien muestra las direcciones de inicio y fin de la misma, solamente permite la edición de la dirección de inicio, ya que la dirección de fin es el propio SP, y el valor mostrado se actualiza automáticamente con los cambios del mismo.

La barra de herramientas del visor de memoria contiene, en la primera línea, un campo de texto para escribir una dirección de memoria y un botón para reiniciar la memoria; y, en la segunda línea, botones de navegación al PC, SP, IX e IY. Al confirmar la entrada de datos en el campo de texto, el foco se sitúa en esa dirección de memoria específica dentro del visor de memoria y, al pulsar el botón de reinicio, todos los valores almacenados en la memoria se restablecen a 0. Los cuatro botones de navegación sitúan el foco en la posición respectiva de cada uno de esos punteros en el visor de memoria.

El visor en sí es una tabla con las siguientes columnas:

- **PC:** Indica si la dirección de memoria de dicha celda está contenida en el PC en ese momento. Si es así, aparece una flecha apuntando hacia la derecha, “señalando” a la dirección de memoria. En caso contrario, la celda queda vacía.
- **Dir:** Indica la dirección de memoria correspondiente a esa celda. Siempre se muestra en formato decimal sin signo o hexadecimal; si se cambia la configuración a decimal con signo, seguirá siendo decimal sin signo (no tiene sentido pasar de direcciones positivas a negativas en mitad de la memoria).
- **Valor:** El valor almacenado dentro de la memoria para esa dirección. Hacer doble clic en cualquier celda de una fila, o pulsar ENTER cuando el foco está en una fila, permite modificar esta celda de la misma, mediante la misma ventana de diálogo que aparece cuando se editan los registros o las direcciones de código y pila.
- **C:** Indicador de pertenencia a la zona de código. Si la dirección asociada a la celda está contenida en la zona de código, aparecerá en la celda un punto negro. De lo contrario, la celda quedará vacía.
- **P:** Indicador de pertenencia a la pila. Si la dirección asociada a la celda está contenida entre el SP (no incluido) y el inicio de la pila, aparecerá en la celda un punto negro. Si el SP apunta a esa dirección, la celda contendrá una flecha orientada a la izquierda. En cualquier otro caso, la celda quedará vacía.
- **IX e IY:** Al igual que la columna del PC, indican si la dirección de memoria está contenida, respectivamente, en los registros IX e IY. Cuando se produzca una coincidencia, la celda contendrá una flecha apuntando a la izquierda, y estará vacía en caso contrario.

11.5. El visor de código

Este componente, que se puede ver con más detalle en la figura 9, se encarga de aportar información sobre el código contenido en la memoria y gestiona los puntos de ruptura. Agrupa los siguientes elementos:

- Módulo de visualización de siguiente instrucción a ejecutar.
- Barra de herramientas del visor.
- El propio visor de código.



Figura 9: Visor de código con su barra de herramientas.

El visualizador de la siguiente instrucción permite mantener en todo momento constancia de la misma, aunque el visor esté desplazado o incluso desfasado. La barra de herramientas, en dos filas como la del visor de memoria, tiene en la primera de ellas un campo de texto para desensamblar el código a partir de una dirección introducida, así como un botón para eliminar todos los puntos de memoria; la segunda contiene un botón que devuelve el foco a la dirección actual del PC y desensambla a partir del mismo, así como dos botones para ‘desfasar’ una posición hacia atrás o hacia delante, desensamblando a partir de la misma.

El sistema de desensamblado sigue estas reglas:

- Al cargar un programa, desensambla todo el código desde la posición inicial de la memoria.
- Es posible navegar mediante scroll hacia adelante cuanto se desee, pero no es posible navegar más atrás de la dirección de desensamblado actual.
- Los botones ‘+1’ y ‘-1’ desplazan hacia delante y hacia atrás, respectivamente, la dirección de desensamblado y, en consecuencia, regeneran la lista de instrucciones.
- El campo de navegación establece la dirección introducida como nueva dirección de desensamblado, regenerando la lista de instrucciones.

- Un cambio en el modo de representación numérica regenerará la lista de instrucciones.
- La lista de instrucciones se regenerará tras cualquier ejecución. Se tomará como dirección de desensamblado el valor actual del PC.

Por otro lado, el visor de código en sí contiene una tabla, similar a la del visor de memoria, con las siguientes columnas:

- **BP:** Indicador de punto de ruptura. Cuando la instrucción tiene asociado un punto de ruptura, en esta celda aparece un círculo negro, quedando vacía en caso contrario. Si alguna de las posiciones de memoria que contienen las palabras que conforman esta instrucción tiene un punto de ruptura asociado, se considera que toda la instrucción lo tiene. Eliminar el punto de ruptura de la instrucción lo elimina de todas las palabras que lo contengan. Crear un punto de ruptura en la instrucción lo establece únicamente en la primera palabra de la misma. Hacer doble clic en cualquier celda de una fila, o pulsar ENTER cuando el foco está en una fila, creará un punto de ruptura o eliminará el existente para esa instrucción.
- **PC:** Indica si el contador de programa apunta actualmente a la primera palabra de la instrucción contenida en la fila. Si es así, aparece una flecha apuntando hacia la derecha, “señalando” a la dirección de memoria. En caso contrario, la celda queda vacía.
- **Dir:** Indica la dirección de memoria correspondiente a esa celda. Siempre se muestra en formato decimal sin signo o hexadecimal; si se cambia la configuración a decimal con signo, seguirá siendo decimal sin signo (igual que en el visor de memoria). El visor no muestra necesariamente todas las direcciones de memoria existentes, solamente las filas cuya dirección sea la primera de una instrucción válida, o bien aquellas que apunten a una dirección inválida.
- **Instrucción:** Contiene la cadena de texto de la instrucción desensamblada a código fuente legible para el usuario.

11.6. Las zonas de texto

Este componente separa los flujos de mensajes del sistema en dos áreas de texto distintas, situadas una encima de la otra: la de arriba se encarga de mostrar todos los mensajes del sistema (avisos de ensamblado exitoso, mensajes de error...), mientras que la segunda gestiona el flujo de entrada y salida de la simulación en curso. Ambas áreas de texto están separadas por un elemento deslizante, que permite redistribuir el espacio vertical asignado a ambas, y crecen horizontalmente con la redimensión de la aplicación.

El área de texto de mensajes del sistema no permite escritura, solamente lectura; y el área de texto de la consola de simulación solamente permite escritura cuando se ejecuta una instrucción ININT, INCHAR o INSTR, que solicita explícitamente una entrada. Además, incluso cuando se permite escribir en la consola, no se permite al usuario editar (borrar, modificar, insertar) el texto ya existente, solamente puede escribir a partir del último carácter del texto previo, y solamente se le permite borrar hasta ese punto.

Además, ambas áreas de texto tienen asociadas ciertas acciones, tanto a través de menú contextual (clic derecho) como mediante atajos de teclado:

- Seleccionar todo (CONTROL+A).
- Copiar selección (CONTROL+C).
- Vaciar de texto la zona (CONTROL+W).

Para evitar comportamientos inesperados, esta última opción está desactivada para el área de la consola mientras está activo el modo de espera de entrada del usuario.

12. Tablas de referencia

12.1. Tablas de formato de instrucciones

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Código de operación										Dir. OP1		Dir. OP2			

(a) Formato de la primera palabra.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP															

(b) Formato de operando de 16 bits.

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								-	-	-	-	-	-	-	-
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	Valor OP2							

(c) Formatos de operando de 8 bits para el primer operando (arriba) y el segundo (abajo).

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				-	-	-	-	-	-	-	-
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	-	-	Valor OP2			

(d) Formatos de operando de 4 bits para el primer operando (arriba) y el segundo (abajo).

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								Valor OP2							
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Valor OP1								-	-	-	-	Valor OP2			
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				Valor OP2							
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	Valor OP1				-	-	-	-	Valor OP2			

(e) Combinaciones de dos operandos en una misma palabra.

Tabla 5: Formato de instrucciones.

12.2. Tabla de modos de direccionamiento, ancho y codificación

Código	Bits	Modo de direccionamiento	Ancho de representación
0	000	Sin operandos	N/A
1	001	Inmediato	16 bits
2	010	Registro	4 bits
3	011	Memoria	16 bits
4	100	Indirecto	4 bits
5	101	Relativo a IX	8 bits
6	110	Relativo a IY	8 bits
7	111	Relativo a PC	8 bits

Tabla 6: Tabla de códigos de direccionamiento.

12.3. Tabla de códigos de registros

Registro	Código	Bits	Registro	Código	Bits
R0	0	0000	R8	8	1000
R1	1	0001	R9	9	1001
R2	2	0010	A	10	1010
R3	3	0011	SR	11	1011
R4	4	0100	IX	12	1100
R5	5	0101	IY	13	1101
R6	6	0110	SP	14	1110
R7	7	0111	PC	15	1111

Tabla 7: Tabla de códigos de registro.

12.4. Tabla de posicionamiento de biestables en el registro de estado

15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
-	-	-	-	-	-	-	-	-	-	H	S	P	V	C	Z

Tabla 8: Tabla de posicionamiento de biestables en el registro SR.

12.5. Tabla de compatibilidad instrucción-direccionamientos

Identificadores		Direccionamiento OP1						Direccionamiento OP2					
Cód.	Inst.	#	.	/	[]	#[]	\$	#	.	/	[]	#[]	\$
0	NOP	X	X	X	X	X	X	X	X	X	X	X	X
1	HALT	X	X	X	X	X	X	X	X	X	X	X	X
2	MOVE	✓	✓	✓	✓	✓	X	X	✓	✓	✓	✓	X
3	PUSH	✓	✓	✓	✓	✓	X	X	X	X	X	X	X
4	POP	X	✓	✓	✓	✓	X	X	X	X	X	X	X
5	ADD	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
6	SUB	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
7	MUL	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
8	DIV	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
9	MOD	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
10	INC	X	✓	✓	✓	✓	X	X	X	X	X	X	X
11	DEC	X	✓	✓	✓	✓	X	X	X	X	X	X	X
12	NEG	X	✓	✓	✓	✓	X	X	X	X	X	X	X
13	CMP	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
14	AND	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
15	OR	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
16	XOR	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
17	NOT	X	✓	✓	✓	✓	X	X	X	X	X	X	X
18	BR	X	X	✓	✓	X	✓	X	X	X	X	X	X
19	BZ	X	X	✓	✓	X	✓	X	X	X	X	X	X
20	BNZ	X	X	✓	✓	X	✓	X	X	X	X	X	X
21	BP	X	X	✓	✓	X	✓	X	X	X	X	X	X
22	BN	X	X	✓	✓	X	✓	X	X	X	X	X	X
23	BV	X	X	✓	✓	X	✓	X	X	X	X	X	X
24	BNV	X	X	✓	✓	X	✓	X	X	X	X	X	X
25	BC	X	X	✓	✓	X	✓	X	X	X	X	X	X
26	BNC	X	X	✓	✓	X	✓	X	X	X	X	X	X
27	BE	X	X	✓	✓	X	✓	X	X	X	X	X	X
28	BO	X	X	✓	✓	X	✓	X	X	X	X	X	X
29	CALL	X	X	✓	✓	X	✓	X	X	X	X	X	X
30	RET	X	X	X	X	X	X	X	X	X	X	X	X
31	INCHAR	X	✓	✓	✓	✓	X	X	X	X	X	X	X
32	ININT	X	✓	✓	✓	✓	X	X	X	X	X	X	X
33	INSTR	X	X	✓	✓	✓	X	X	X	X	X	X	X
34	WRCHAR	✓	✓	✓	✓	✓	X	X	X	X	X	X	X
35	WRINT	✓	✓	✓	✓	✓	X	X	X	X	X	X	X
36	WRSTR	X	X	✓	✓	✓	X	X	X	X	X	X	X

Tabla 9: Tabla de modos de direccionamiento aceptados en cada operando por cada instrucción.